Hospital Patient Analysis: Without NumPy vs With NumPy

1. Storing data

Without NumPy — Python lists

patients = ["Ravi", "Priya", "Rahul", "Sneha", "Arjun"]age = [25, 30, 45, 35, 50]bill = [15000, 20000, 10000, 25000, 30000]

We store data in normal Python lists.

With NumPy — NumPy arrays

import numpy as nppatients = np.array(["Ravi", "Priya", "Rahul", "Sneha", "Arjun"])age = np.array([25, 30, 45, 35, 50])bill = np.array([15000, 20000, 10000, 25000, 30000])

NumPy provides arrays designed for efficient numerical operations.

2. Speed and performance

Without NumPy: We often use Python loops to process numerical values one by one, which can add overhead for large datasets.

total = 0for b in bill: total += bprint(total)

With NumPy: We use optimized numerical operations on the whole array.

print(np.sum(bill))

Both produce `100000`. NumPy is often faster for large numerical workloads, although the exact speed depends on the operation and dataset size.

3. Vectorization

Without NumPy: Use a loop to multiply each hospital bill by two.

bill = [15000, 20000, 10000, 25000, 30000]result = []for b in bill: result.append(b * 2)print(result)

With NumPy: Multiply every bill directly.

bill = np.array([15000, 20000, 10000, 25000, 30000])print(bill * 2)

NumPy applies the operation element-wise without an explicit Python loop.

4. Built-in mathematical functions

Without NumPy: Use loops and Python's built-in functions.

total = 0for b in bill: total += baverage = total / len(bill)highest = max(bill)lowest = min(bill)print(total, average, highest, lowest)

With NumPy: Use ready-made numerical functions.

print(np.sum(bill))print(np.mean(bill))print(np.max(bill))print(np.min(bill))

The results are:

- Total bill: ₹1,00,000
- Average bill: ₹20,000
- Highest bill: ₹30,000
- Lowest bill: ₹10,000

5. Filtering patients

Without NumPy: Use a loop and an `if` condition.

for i in range(len(patients)): if bill[i] > 10000: print(patients[i], bill[i])

With NumPy: Use Boolean indexing.

result = patients[bill > 10000]print(result)

This returns Ravi, Priya, Sneha, and Arjun.

6. Average patient age

Without NumPy: Calculate the total age using a loop and divide by the number of patients.

age = [25, 30, 45, 35, 50]total_age = 0for a in age: total_age += aaverage_age = total_age / len(age)print(average_age)

With NumPy: Use `np.mean()`.

age = np.array([25, 30, 45, 35, 50])print(np.mean(age))

Both return `37.0`.

7. Memory efficiency

Without NumPy: Python lists store references to Python objects, which can create additional memory overhead for large collections of numbers.

With NumPy: Numeric arrays typically store values in a compact, homogeneous memory buffer, making them more memory-efficient for many numerical workloads.

8. Multidimensional data

Without NumPy: We can use nested lists, but operations on rows and columns often require nested loops.

bills = [[15000, 20000],   [10000, 25000]]

With NumPy: Create a 2D array and calculate row or column totals directly.

bills = np.array([[15000, 20000],   [10000, 25000]])print(bills.sum(axis=1)) # Row totalsprint(bills.sum(axis=0)) # Column totals

9. Complex numerical operations

Without NumPy: Matrix calculations, element-wise operations, and multidimensional numerical processing generally require more manual code.

With NumPy: We can use array operations, broadcasting, matrix multiplication, and linear algebra functions.

a = np.array([1, 2, 3])b = np.array([4, 5, 6])print(a * b) # Element-wise multiplicationprint(a @ b) # Dot product

10. Important limitation

NumPy is primarily designed for numerical arrays. For hospital records containing patient names, ages, departments, and bills, Pandas is generally more convenient for structured data analysis.

For example, calculating average bills by department is straightforward with Pandas:

df.groupby("Department")["Bill"].mean()

Conclusion: Without NumPy, we often rely on Python lists, loops, and conditions. With NumPy, numerical calculations become more concise and can be more efficient through vectorization and optimized array operations. Pandas complements NumPy by providing tools for working with tabular records, filtering patients, and analyzing departments.